

CS 224n: Assignment #4

This assignment is split into two sections: *Neural Machine Translation with RNNs* and *Analyzing NMT Systems*. The first is primarily coding and implementation focused, whereas the second entirely consists of written, analysis questions. If you get stuck on the first section, you can always work on the second as the two sections are independent of each other. Note that the NMT system is more complicated than the neural networks we have previously constructed within this class and takes about **2 hours to train on a GPU**. Thus, we strongly recommend you get started early with this assignment. Finally, the notation and implementation of the NMT system is a bit tricky, so if you ever get stuck along the way, please come to Office Hours so that the TAs can support you.

1. Neural Machine Translation with RNNs (45 points)

In Machine Translation, our goal is to convert a sentence from the *source* language (e.g. Mandarin Chinese) to the *target* language (e.g. English). In this assignment, we will implement a sequence-to-sequence (Seq2Seq) network with attention, to build a Neural Machine Translation (NMT) system. In this section, we describe the **training procedure** for the proposed NMT system, which uses a Bidirectional LSTM Encoder and a Unidirectional LSTM Decoder.

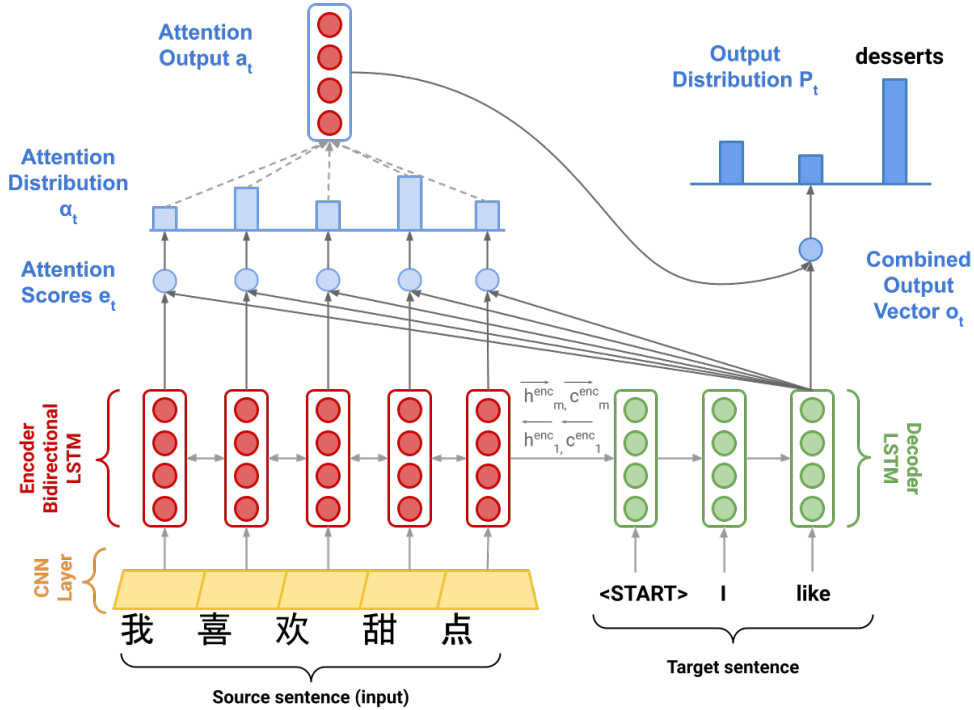


Figure 1: Seq2Seq Model with Multiplicative Attention, shown on the third step of the decoder. Hidden states $\vec{h}_i^{enc}$ and cell states $\vec{c}_i^{enc}$ are defined on the next page.

Model description (training procedure)

Given a sentence in the source language, we look up the character or word embeddings from an **embeddings matrix**, yielding $\mathbf{x}_1, \dots, \mathbf{x}_m$ ($\mathbf{x}_i \in \mathbb{R}^{e \times 1}$), where m is the length of the source sentence and e is

the embedding size. We then feed the embeddings to a **convolutional layer**¹ while maintaining their shapes. We feed the convolutional layer outputs to the **bidirectional encoder**, yielding hidden states and cell states for both the forwards ($\rightarrow$) and backwards ($\leftarrow$) LSTMs. The forwards and backwards versions are concatenated to give hidden states $\mathbf{h}_i^{\text{enc}}$ and cell states $\mathbf{c}_i^{\text{enc}}$:

$$\mathbf{h}_i^{\text{enc}} = [\overleftarrow{\mathbf{h}_i^{\text{enc}}}, \overrightarrow{\mathbf{h}_i^{\text{enc}}}] \text{ where } \mathbf{h}_i^{\text{enc}} \in \mathbb{R}^{2h \times 1}, \overleftarrow{\mathbf{h}_i^{\text{enc}}}, \overrightarrow{\mathbf{h}_i^{\text{enc}}} \in \mathbb{R}^{h \times 1} \quad 1 \leq i \leq m \quad (1)$$

$$\mathbf{c}_i^{\text{enc}} = [\overleftarrow{\mathbf{c}_i^{\text{enc}}}, \overrightarrow{\mathbf{c}_i^{\text{enc}}}] \text{ where } \mathbf{c}_i^{\text{enc}} \in \mathbb{R}^{2h \times 1}, \overleftarrow{\mathbf{c}_i^{\text{enc}}}, \overrightarrow{\mathbf{c}_i^{\text{enc}}} \in \mathbb{R}^{h \times 1} \quad 1 \leq i \leq m \quad (2)$$

We then initialize the **decoder's** first hidden state $\mathbf{h}_0^{\text{dec}}$ and cell state $\mathbf{c}_0^{\text{dec}}$ with a linear projection of the encoder's final hidden state and final cell state.²

$$\mathbf{h}_0^{\text{dec}} = \mathbf{W}_h [\overleftarrow{\mathbf{h}_1^{\text{enc}}}, \overrightarrow{\mathbf{h}_m^{\text{enc}}}] \text{ where } \mathbf{h}_0^{\text{dec}} \in \mathbb{R}^{h \times 1}, \mathbf{W}_h \in \mathbb{R}^{h \times 2h} \quad (3)$$

$$\mathbf{c}_0^{\text{dec}} = \mathbf{W}_c [\overleftarrow{\mathbf{c}_1^{\text{enc}}}, \overrightarrow{\mathbf{c}_m^{\text{enc}}}] \text{ where } \mathbf{c}_0^{\text{dec}} \in \mathbb{R}^{h \times 1}, \mathbf{W}_c \in \mathbb{R}^{h \times 2h} \quad (4)$$

With the decoder initialized, we must now feed it a target sentence. On the t^{th} step, we look up the embedding for the t^{th} subword, $\mathbf{y}_t \in \mathbb{R}^{e \times 1}$. We then concatenate $\mathbf{y}_t$ with the *combined-output vector* $\mathbf{o}_{t-1} \in \mathbb{R}^{h \times 1}$ from the previous timestep (we will explain what this is later down this page!) to produce $\overline{\mathbf{y}}_t \in \mathbb{R}^{(e+h) \times 1}$. Note that for the first target subword (i.e. the start token) $\mathbf{o}_0$ is a zero-vector. We then feed $\overline{\mathbf{y}}_t$ as input to the decoder.

$$\mathbf{h}_t^{\text{dec}}, \mathbf{c}_t^{\text{dec}} = \text{Decoder}(\overline{\mathbf{y}}_t, \mathbf{h}_{t-1}^{\text{dec}}, \mathbf{c}_{t-1}^{\text{dec}}) \text{ where } \mathbf{h}_t^{\text{dec}} \in \mathbb{R}^{h \times 1}, \mathbf{c}_t^{\text{dec}} \in \mathbb{R}^{h \times 1} \quad (5)$$

$$(6)$$

We then use $\mathbf{h}_t^{\text{dec}}$ to compute multiplicative attention over $\mathbf{h}_1^{\text{enc}}, \dots, \mathbf{h}_m^{\text{enc}}$:

$$\mathbf{e}_{t,i} = (\mathbf{h}_t^{\text{dec}})^T \mathbf{W}_{\text{attProj}} \mathbf{h}_i^{\text{enc}} \text{ where } \mathbf{e}_t \in \mathbb{R}^{m \times 1}, \mathbf{W}_{\text{attProj}} \in \mathbb{R}^{h \times 2h} \quad 1 \leq i \leq m \quad (7)$$

$$\alpha_t = \text{softmax}(\mathbf{e}_t) \text{ where } \alpha_t \in \mathbb{R}^{m \times 1} \quad (8)$$

$$\mathbf{a}_t = \sum_{i=1}^m \alpha_{t,i} \mathbf{h}_i^{\text{enc}} \text{ where } \mathbf{a}_t \in \mathbb{R}^{2h \times 1} \quad (9)$$

$\mathbf{e}_{t,i}$ is a scalar, the i th element of $\mathbf{e}_t \in \mathbb{R}^{m \times 1}$, computed using the hidden state of the decoder at the t th step, $\mathbf{h}_t^{\text{dec}} \in \mathbb{R}^{h \times 1}$, the attention projection $\mathbf{W}_{\text{attProj}} \in \mathbb{R}^{h \times 2h}$, and the hidden state of the encoder at the i th step, $\mathbf{h}_i^{\text{enc}} \in \mathbb{R}^{2h \times 1}$.

We now concatenate the attention output $\mathbf{a}_t$ with the decoder hidden state $\mathbf{h}_t^{\text{dec}}$ and pass this through a linear layer, tanh, and dropout to attain the *combined-output vector* $\mathbf{o}_t$.

$$\mathbf{u}_t = [\mathbf{a}_t; \mathbf{h}_t^{\text{dec}}] \text{ where } \mathbf{u}_t \in \mathbb{R}^{3h \times 1} \quad (10)$$

$$\mathbf{v}_t = \mathbf{W}_u \mathbf{u}_t \text{ where } \mathbf{v}_t \in \mathbb{R}^{h \times 1}, \mathbf{W}_u \in \mathbb{R}^{h \times 3h} \quad (11)$$

$$\mathbf{o}_t = \text{dropout}(\tanh(\mathbf{v}_t)) \text{ where } \mathbf{o}_t \in \mathbb{R}^{h \times 1} \quad (12)$$

¹Checkout <https://cs231n.github.io/convolutional-networks> for an in-depth description for convolutional layers if you are not familiar

²If it's not obvious, think about why we regard $[\overleftarrow{\mathbf{h}_1^{\text{enc}}}, \overrightarrow{\mathbf{h}_m^{\text{enc}}}]$ as the 'final hidden state' of the Encoder.

Then, we produce a probability distribution $\mathbf{P}_t$ over target subwords at the t^{th} timestep:

$$\mathbf{P}_t = \text{softmax}(\mathbf{W}_{\text{vocab}} \mathbf{o}_t) \text{ where } \mathbf{P}_t \in \mathbb{R}^{V_t \times 1}, \mathbf{W}_{\text{vocab}} \in \mathbb{R}^{V_t \times h} \quad (13)$$

Here, V_t is the size of the target vocabulary. Finally, to train the network we then compute the cross entropy loss between $\mathbf{P}_t$ and $\mathbf{g}_t$, where $\mathbf{g}_t$ is the one-hot vector of the target subword at timestep t :

$$J_t(\theta) = \text{CrossEntropy}(\mathbf{P}_t, \mathbf{g}_t) \quad (14)$$

Here, θ represents all the parameters of the model and $J_t(\theta)$ is the loss on step t of the decoder. Now that we have described the model, let's try implementing it for Mandarin Chinese to English translation!

Setting up your Virtual Machine

Follow the instructions in the [CS224n Azure Guide](#) (link also provided on website and Ed) in order to create your VM instance. This should take you approximately 45 minutes. Though you will need the GPU to train your model, we strongly advise that you first develop the code locally and ensure that it runs, before attempting to train it on your VM. GPU time is expensive and limited. It takes approximately **1.5 to 2 hours** to train the NMT system. We don't want you to accidentally use all your GPU time for debugging your model rather than training and evaluating it. Finally, **make sure that your VM is turned off whenever you are not using it.**

If your Azure subscription runs out of money, your VM will be temporarily locked and inaccessible. If that happens, please fill out a request form [here](#).

In order to run the model code on your **local** machine, please run the following command to create the proper virtual environment:

```
conda env create --file local_env.yml
```

Note that this virtual environment **will not** be needed on the VM.

Implementation and written questions

- (a) (2 points) (coding) In order to apply tensor operations, we must ensure that the sentences in a given batch are of the same length. Thus, we must identify the longest sentence in a batch and pad others to be the same length. Implement the `pad_sents` function in `utils.py`, which shall produce these padded sentences.
- (b) (3 points) (coding) Implement the `__init__` function in `model_embeddings.py` to initialize the necessary source and target embeddings.
- (c) (4 points) (coding) Implement the `__init__` function in `nmt_model.py` to initialize the necessary model layers (LSTM, CNN, projection, and dropout) for the NMT system.
- (d) (8 points) (coding) Implement the `encode` function in `nmt_model.py`. This function converts the padded source sentences into the tensor $\mathbf{X}$, generates $\mathbf{h}_1^{\text{enc}}, \dots, \mathbf{h}_m^{\text{enc}}$, and computes the initial state $\mathbf{h}_0^{\text{dec}}$ and initial cell $\mathbf{c}_0^{\text{dec}}$ for the Decoder. You can run a non-comprehensive sanity check by executing:

```
python sanity_check.py 1d
```

- (e) (8 points) (coding) Implement the decode function in `nmt_model.py`. This function constructs $\bar{\mathbf{y}}$ and runs the step function over every timestep for the input. You can run a non-comprehensive sanity check by executing:

```
python sanity_check.py 1e
```

- (f) (10 points) (coding) Implement the step function in `nmt_model.py`. This function applies the Decoder's LSTM cell for a single timestep, computing the encoding of the target subword $\mathbf{h}_t^{\text{dec}}$, the attention scores $\mathbf{e}_t$, attention distribution α_t , the attention output $\mathbf{a}_t$, and finally the combined output $\mathbf{o}_t$. You can run a non-comprehensive sanity check by executing:

```
python sanity_check.py 1f
```

- (g) (3 points) (written) The `generate_sent_masks()` function in `nmt_model.py` produces a tensor called `enc_masks`. It has shape (batch size, max source sentence length) and contains 1s in positions corresponding to 'pad' tokens in the input, and 0s for non-pad tokens. Look at how the masks are used during the attention computation in the `step()` function (lines 311-312).

First explain (in around three sentences) what effect the masks have on the entire attention computation. Then explain (in one or two sentences) why it is necessary to use the masks in this way.

Solution: The masks are used to set the previously computed attention scores $e_{t,i}$ of the padding tokens to $-\infty$ right before the softmax layer. Consequently, when the softmax function is applied, these extremely negative attention scores map precisely to an attention probability $\alpha_{t,i}$ of 0. This ensures that when computing the final context vector $\mathbf{a}_t$ via a weighted sum, the encoder hidden states corresponding to padding tokens contribute absolutely nothing to the result. It is necessary to use masks in this way because padding tokens are artificial fillers added solely to equalize sentence lengths within a batch and possess no real semantic meaning. If they were not masked out, the padding tokens would receive non-zero attention weights, injecting meaningless noise into the decoder's translation process.

Now it's time to get things running! As noted earlier, we recommend that you develop the code on your personal computer. Confirm that you are running in the proper conda environment and then execute the following command to train the model on your local machine:

```
sh run.sh train_local
(Windows) run.bat train_local
```

For a faster way to debug by training on less data, you can run the following instead:

```
sh run.sh train_debug
(Windows) run.bat debug
```

To help with monitoring and debugging, the starter code uses tensorboard to log loss and perplexity during training using TensorBoard³. TensorBoard provides tools for logging and visualizing training information from experiments. To open TensorBoard, run the following in your conda environment:

```
tensorboard --logdir=runs
```

You should see a significant decrease in loss during the initial iterations. Once you have ensured that your code does not crash (i.e. let it run till iter 10 or iter 20), power on your VM from the Azure Web

³<https://pytorch.org/docs/stable/tensorboard.html>

Portal. Then read the *Managing Code Deployment to a VM* section of our [Practical Guide to VMs](#) (link also given on website and Ed) for instructions on how to upload your code to the VM.

Next, install necessary packages to your VM by running:

```
pip install -r gpu_requirements.txt
```

Finally, turn to the *Managing Processes on a VM* section of the Practical Guide and follow the instructions to create a new tmux session. Concretely, run the following command to create tmux session called nmt.

```
tmux new -s nmt
```

Once your VM is configured and you are in a tmux session, execute:

```
sh run.sh train
(Windows) run.bat train
```

Once you know your code is running properly, you can detach from session and close your ssh connection to the server. To detach from the session, run:

```
tmux detach
```

You can return to your training model by ssh-ing back into the server and attaching to the tmux session by running:

```
tmux a -t nmt
```

- (h) (3 points) (written) Once your model is done training (**this should take under 2 hours on the VM**), execute the following command to test the model:

```
sh run.sh test
(Windows) run.bat test
```

Please report the model's BLEU Score. It should be larger than 18.

Solution: The model's corpus BLEU Score is 19.899.

- (i) (4 points) (written) In class, we learned about dot product attention, multiplicative attention, and additive attention. As a reminder, dot product attention is $\mathbf{e}_{t,i} = \mathbf{s}_t^T \mathbf{h}_i$, multiplicative attention is $\mathbf{e}_{t,i} = \mathbf{s}_t^T \mathbf{W} \mathbf{h}_i$, and additive attention is $\mathbf{e}_{t,i} = \mathbf{v}^T \tanh(\mathbf{W}_1 \mathbf{h}_i + \mathbf{W}_2 \mathbf{s}_t)$.
- (2 points) Explain one advantage and one disadvantage of *dot product attention* compared to multiplicative attention.
 - (2 points) Explain one advantage and one disadvantage of *additive attention* compared to multiplicative attention.

Solution:

- **Dot product vs Multiplicative:**

- **Advantage:** Dot product attention operates parameter-free without needing to learn an extra weight matrix $\mathbf{W}$. This makes it computationally faster and highly space-efficient.
- **Disadvantage:** Dot product attention strictly requires the encoder hidden state $\mathbf{h}_i$ and decoder hidden state $\mathbf{s}_t$ to have the exact same dimensionality. Multiplicative attention avoids this limitation since the weight matrix $\mathbf{W}$ can be sized to project representations of different dimensions.

- **Additive vs Multiplicative:**

- **Advantage:** Additive attention tends to be more stable when the dimensionality is very large. In multiplicative attention, large dimensions can result in dot products with large variance, pushing the softmax function into regions with extremely small gradients. Additive attention also provides a non-linear mapping (via $\tanh$), which can capture more complex relationships.
- **Disadvantage:** Additive attention is computationally more expensive and less memory-efficient. Multiplicative attention can be implemented via highly optimized, easily parallelizable matrix multiplications, whereas additive attention involves broadcasting, computing non-linearities, and learning multiple weight matrices ($\mathbf{W}_1$, $\mathbf{W}_2$, $\mathbf{v}$).

2. Analyzing NMT Systems (25 points)

- (a) (3 points) Look at the `src.vocab` file for some examples of phrases and words in the source language vocabulary. When encoding an input Mandarin Chinese sequence into “pieces” in the vocabulary, the tokenizer maps the sequence to a series of vocabulary items, each consisting of one or more characters (thanks to the `sentencepiece` tokenizer, we can perform this segmentation even when the original text has no white space). Given this information, how could adding a 1D Convolutional layer after the embedding layer and before passing the embeddings into the bidirectional encoder help our NMT system? **Hint:** each Mandarin Chinese character is either an entire word or a morpheme in a word. Look up the meanings of 电, 脑, and 电脑 separately for an example. The characters 电 (electricity) and 脑 (brain) when combined into the phrase 电脑 mean computer.

Solution: Adding a 1D Convolutional layer after the embedding layer allows the model to capture local context and combine the representations of adjacent characters or subwords (n-grams) into higher-level semantic features before they reach the encoder. In Mandarin Chinese, single characters are often morphemes that combine to form multi-character words with distinct or compositional meanings (such as “电” and “脑” combining into “电脑”). A 1D CNN with a sliding window across the sequence can effectively explicitly construct and extract these local compositional features (multi-character words or phrases) regardless of their position in the sentence. This provides the bidirectional encoder with a much richer, word-level structural representation rather than treating each isolated character entirely disjointly.

- (b) (8 points) Here we present a series of errors we found in the outputs of our NMT model (which is the same as the one you just trained). For each example of a reference (i.e., ‘gold’) English translation, and NMT (i.e., ‘model’) English translation, please:
1. Identify the error in the NMT translation.
 2. Provide possible reason(s) why the model may have made the error (either due to a specific linguistic construct or a specific model limitation).
 3. Describe one possible way we might alter the NMT system to fix the observed error. There are more than one possible fixes for an error. For example, it could be tweaking the size of the hidden layers or changing the attention mechanism.

Below are the translations that you should analyze as described above. Only analyze the underlined error in each sentence. Rest assured that you don’t need to know Mandarin to answer these questions. You just need to know English! If, however, you would like some additional color on the source sentences, feel free to use a resource like https://www.archchinese.com/chinese_english_

[dictionary.html](#) to look up words. Feel free to search the training data file to have a better sense of how often certain characters occur.

- i. (2 points) **Source Sentence:** 贼人其后被警方拘捕及被判处盗窃罪名成立。

Reference Translation: *the culprits were subsequently arrested and convicted.*

NMT Translation: *the culprit was subsequently arrested and sentenced to theft.*

Solution:

1. **Error:** The model generated the singular subject “the culprit was” instead of the plural “the culprits were”.
2. **Reason:** Mandarin Chinese nouns typically lack explicit morphological plural markers (e.g., “贼人” translates to either a single thief or multiple thieves). The model lacked the broader context to infer plurality and defaulted to the more statistically common singular form.
3. **Fix:** Incorporate document-level context (translating multi-sentence windows instead of isolated sentences) so the model can implicitly capture semantic hints about plurality from surrounding text.

- ii. (2 points) **Source Sentence:** 几乎已经没有地方容纳这些人, 资源已经用尽。

Reference Translation: *there is almost no space to accommodate these people, and resources have run out.*

NMT Translation: *the resources have been exhausted and resources have been exhausted.*

Solution:

1. **Error:** The NMT translation dropped the first half of the source sentence and incorrectly stuttered/repeated the phrase “resources have been exhausted”.
2. **Reason:** This is a common issue in sequence-to-sequence models with attention architectures, known as over-translation and under-translation. The attention mechanism likely got stuck on the last few tokens of the source sentence (“资源已经用尽”) and failed to traverse the rest of the sentence.
3. **Fix:** Incorporate an attention coverage penalty into the loss function or during inference (e.g., in beam search). This penalizes the model for attending to the same source words repeatedly, forcing the attention distribution to scan comprehensively across the entire original sentence.

- iii. (2 points) **Source Sentence:** 当局已经宣布今天是国殇日。

Reference Translation: *authorities have announced a national mourning today.*

NMT Translation: *the administration has announced today's day.*

Solution:

1. **Error:** The model failed to translate the concept of “国殇日” (national mourning day) entirely, collapsing it into the awkward phrase “today’s day”.
2. **Reason:** “国殇日” is likely a rare out-of-vocabulary (OOV) compound word in the training corpus. Because the model didn’t learn a strong representation for it, it ignored the word and hallucinated an output based on the surrounding familiar tokens like “今天” (today) and “是” (is).
3. **Fix:** Use a finer subword tokenization strategy (e.g., smaller BPE vocabulary size) to ensure rare compound words are mechanically broken down into recognizable characters (e.g., “国” nation, “殇” mourn, “日” day) so the model can compositionally translate them.

- iv. (2 points) **Source Sentence**⁴: 俗语有云:“唔做唔错”。

Reference Translation: “*act not, err not*”, so a saying goes.

NMT Translation: *as the saying goes, “it’s not wrong.”*

Solution:

1. **Error:** The model wrongly translates the Cantonese colloquial phrase “唔做唔错” (act not, err not) as “it’s not wrong”.
2. **Reason:** The phrase originates from a regional dialect (Cantonese). Because the NMT model was primarily trained on standard mainland news (which lacks such phrases), it didn’t recognize the structure. It merely recognized “唔” (not) and “错” (wrong) and hallucinated the translation.
3. **Fix:** Augment the training data with a conversational and dialectal corpus that includes Cantonese phrases. Alternatively, implement a back-translation strategy on Cantonese data to help the model learn cross-dialect mapping.

- (c) (14 points) BLEU score is the most commonly used automatic evaluation metric for NMT systems. It is usually calculated across the entire test set, but here we will consider BLEU defined for a single example.⁵ Suppose we have a source sentence $\mathbf{s}$, a set of k reference translations $\mathbf{r}_1, \dots, \mathbf{r}_k$, and a candidate translation $\mathbf{c}$. To compute the BLEU score of $\mathbf{c}$, we first compute the *modified n -gram precision* p_n of $\mathbf{c}$, for each of $n = 1, 2, 3, 4$, where n is the n in **n-gram**:

$$p_n = \frac{\sum_{\text{ngram} \in \mathbf{c}} \min \left(\max_{i=1, \dots, k} \text{Count}_{\mathbf{r}_i}(\text{ngram}), \text{Count}_{\mathbf{c}}(\text{ngram}) \right)}{\sum_{\text{ngram} \in \mathbf{c}} \text{Count}_{\mathbf{c}}(\text{ngram})} \quad (15)$$

Here, for each of the n -grams that appear in the candidate translation $\mathbf{c}$, we count the maximum number of times it appears in any one reference translation, capped by the number of times it appears in $\mathbf{c}$ (this is the numerator). We divide this by the number of n -grams in $\mathbf{c}$ (denominator).

Next, we compute the *brevity penalty* BP. Let $\text{len}(\mathbf{c})$ be the length of $\mathbf{c}$ and let $\text{len}(\mathbf{r})$ be the length of the reference translation that is closest to $\text{len}(\mathbf{c})$ (in the case of two equally-close reference translation lengths, choose $\text{len}(\mathbf{r})$ as the shorter one).

$$BP = \begin{cases} 1 & \text{if } \text{len}(\mathbf{c}) \geq \text{len}(\mathbf{r}) \\ \exp \left(1 - \frac{\text{len}(\mathbf{r})}{\text{len}(\mathbf{c})} \right) & \text{otherwise} \end{cases} \quad (16)$$

Lastly, the BLEU score for candidate $\mathbf{c}$ with respect to $\mathbf{r}_1, \dots, \mathbf{r}_k$ is:

$$BLEU = BP \times \exp \left(\sum_{n=1}^4 \lambda_n \log p_n \right) \quad (17)$$

where $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ are weights that sum to 1. The log here is natural log.

⁴This is a Cantonese sentence! The data used in this assignment comes from GALE Phase 3, which is a compilation of news written in simplified Chinese from various sources scraped from the internet along with their translations. For more details, see <https://catalog.ldc.upenn.edu/LDC2017T02>.

⁵This definition of sentence-level BLEU score matches the `sentence_bleu()` function in the nltk Python package. Note that the NLTK function is sensitive to capitalization. In this question, all text is lowercased, so capitalization is irrelevant. http://www.nltk.org/api/nltk.translate.html#nltk.translate.bleu_score.sentence_bleu

- i. (5 points) Please consider this example:

Source Sentence s : 需要有充足和可预测的资源。

Reference Translation r_1 : *resources have to be sufficient and they have to be predictable*

Reference Translation r_2 : *adequate and predictable resources are required*

NMT Translation c_1 : there is a need for adequate and predictable resources

NMT Translation c_2 : resources be sufficient and predictable to

Please compute the BLEU scores for c_1 and c_2 . Let $\lambda_i = 0.5$ for $i \in \{1, 2\}$ and $\lambda_i = 0$ for $i \in \{3, 4\}$ (**this means we ignore 3-grams and 4-grams**, i.e., don't compute p_3 or p_4).

When computing BLEU scores, show your work (i.e., show your computed values for p_1 , p_2 , $\text{len}(c)$, $\text{len}(r)$ and BP). Note that the BLEU scores can be expressed between 0 and 1 or between 0 and 100. The code is using the 0 to 100 scale while in this question we are using the **0 to 1** scale. Please round your responses to 3 decimal places.

Which of the two NMT translations is considered the better translation according to the BLEU Score? Do you agree that it is the better translation?

Solution: For NMT Translation c_1 :

- $\text{len}(c_1) = 9$. The references have lengths $\text{len}(r_1) = 11$ and $\text{len}(r_2) = 6$. The closer length is 11, therefore $\text{len}(r) = 11$.
- $BP = \exp(1 - \frac{11}{9}) = e^{-2/9} \approx 0.801$
- $p_1 = \frac{4}{9}$ (Matches 4 unigrams: *adequate, and, predictable, resources*)
- $p_2 = \frac{3}{8}$ (Matches 3 bigrams: *adequate and, and predictable, predictable resources*)
- $\text{BLEU}_{c_1} = 0.801 \times \exp(0.5 \ln \frac{4}{9} + 0.5 \ln \frac{3}{8}) = 0.801 \times \sqrt{\frac{1}{6}} \approx 0.8007 \times 0.4082 \approx 0.327$

For NMT Translation c_2 :

- $\text{len}(c_2) = 6$. The closer length is $\text{len}(r_2) = 6$, therefore $\text{len}(r) = 6$.
- $BP = 1$ (because $\text{len}(c_2) \geq \text{len}(r)$)
- $p_1 = \frac{6}{6} = 1$ (Matches 6 unigrams: *resources, be, sufficient, and, predictable, to*)
- $p_2 = \frac{3}{5}$ (Matches 3 bigrams: *be sufficient, sufficient and, and predictable*)
- $\text{BLEU}_{c_2} = 1 \times \exp(0.5 \ln 1 + 0.5 \ln \frac{3}{5}) = 1 \times \sqrt{0.6} \approx 0.775$

Conclusion: According to the scores, NMT c_2 (0.775) is currently considered the better translation. I **do not agree** with this evaluation. c_1 is completely grammatically correct and accurately represents the source sentence's meaning. c_2 produces an ungrammatical fragment. c_2 's BLEU was unnaturally inflated because it simultaneously minimized the brevity penalty using r_2 , while selectively piecing together arbitrary matching n -grams independently distributed across both r_1 and r_2 .

- ii. (5 points) Our hard drive was corrupted and we lost Reference Translation r_1 . Please recompute BLEU scores for c_1 and c_2 , this time with respect to r_2 only. Which of the two NMT translations now receives the higher BLEU score? Do you agree that it is the better translation?

Solution: With only reference r_2 (*adequate and predictable resources are required*, length 6) available:

For NMT Translation c_1 :

- $\text{len}(c_1) = 9$. $\text{len}(r) = 6$.
- $BP = 1$ (because $9 > 6$)
- $p_1 = \frac{4}{9}$ (Matches: *adequate, and, predictable, resources*)
- $p_2 = \frac{3}{8}$ (Matches: *adequate and, and predictable, predictable resources*)

- $\text{BLEU}_{c_1} = 1 \times \sqrt{\frac{4}{9} \times \frac{3}{8}} = \sqrt{\frac{1}{6}} \approx 0.408$

For NMT Translation c_2 :

- $\text{len}(c_2) = 6$. $\text{len}(r) = 6$.
- $BP = 1$
- $p_1 = \frac{3}{6} = 0.5$ (Matches: *resources*, *and*, *predictable*)
- $p_2 = \frac{1}{5} = 0.2$ (Matches: *and* *predictable*)
- $\text{BLEU}_{c_2} = 1 \times \sqrt{0.5 \times 0.2} = \sqrt{0.1} \approx 0.316$

Conclusion: NMT Translation c_1 (0.408) now receives a higher BLEU score than c_2 (0.316). I agree with this result, translation c_1 conveys a much better response. Without r_1 's vocabulary, the metric correctly exposed c_2 as the inferior partial translation.

- iii. (2 points) Due to data availability, NMT systems are often evaluated with respect to only a single reference translation. Please explain (in a few sentences) why this may be problematic. In your explanation, discuss how the BLEU score metric assesses the quality of NMT translations when there are multiple reference translations versus a single reference translation.

Solution: BLEU purely measures structural precision by matching string n -grams. However, language is highly flexible; there are numerous valid grammatical synonyms and phrasings available for translating a single source sentence. If we use only a single reference, an NMT model might produce an objectively perfect sentence that is severely penalized because it happens to employ a different vocabulary variant than the reference writer chose. Having multiple references naturally injects diverse language representations and length variances, ensuring BLEU captures and credits various valid translation patterns instead of heavily skewing toward one subjective phrasing standard.

- iv. (2 points) List two advantages and two disadvantages of BLEU, compared to human evaluation, as an evaluation metric for Machine Translation.

Solution:

- **Advantages:** 1. It is fully automated, rapid to compute, and virtually free. This enables engineering teams to instantly score model improvements and hyperparameter changes without waiting for human linguistic experts. 2. It provides a purely objective, highly reproducible, and standardized metric across different scientific cohorts devoid of the inherent subjectivity of human translation bias.
- **Disadvantages:** 1. It lacks semantic understanding and completely ignores logical grammar or fluency. A jumbled ungrammatical sentence possessing exactly matching tokens could outscore a fluidly coherent translation utilizing slightly different vocabulary features (as seen in the earlier c_2 vs c_1 example). 2. Because humans frequently paraphrase sentences via structurally disjoint idioms, evaluating string n -gram exact overlaps systematically fails to identify or reward correct paraphrases and semantically valid synonyms.

Submission Instructions

You shall submit this assignment on GradeScope as two submissions – one for “Assignment 4 [coding]” and another for ‘Assignment 4 [written]’:

1. Run the `collect_submission.sh` script on Azure to produce your `assignment4.zip` file. You can use [scp](#) to transfer files between Azure and your local computer.

2. Upload your assignment4.zip file to GradeScope to “Assignment 4 [coding]”.
3. Upload your written solutions to GradeScope to “Assignment 4 [written]”. When you submit your assignment, make sure to tag all the pages for each problem according to Gradescope’s submission directions. Points will be deducted if the submission is not correctly tagged.